

MC521 - Relatório VIII

Júlio da Silva Telles

22 de Junho de 2026

1 Tasks and Deadlines (CSES - 1630)

O problema propõe um cenário com N tarefas a serem processadas. Cada tarefa possui uma duração específica e um prazo final (deadline). A recompensa obtida ao finalizar uma tarefa é calculada subtraindo o tempo em que ela foi concluída de seu prazo final. O objetivo é determinar a ordem de execução das tarefas que maximize a recompensa total acumulada.

1.1 Solução

Este problema pode ser resolvido de forma eficiente utilizando um Algoritmo Guloso (*Greedy Algorithm*). A lógica fundamental é que processar tarefas mais curtas primeiro minimiza o tempo de espera das tarefas subsequentes, maximizando a pontuação geral. A complexidade de tempo é dominada pela ordenação, resultando em $\mathcal{O}(N \log N)$. O algoritmo segue os seguintes passos:

1. **Armazenamento de Dados:** Lemos as entradas e armazenamos a duração e o prazo de cada tarefa em um vetor de pares (`std::pair`), onde o primeiro elemento (`first`) representa a duração e o segundo (`second`) representa o deadline.
2. **Estratégia Gulosa (Ordenação):** Utilizamos a função `sort` nativa da linguagem para ordenar o vetor. Como armazenamos a duração no primeiro elemento do par, a ordenação ocorre automaticamente de forma crescente com base no tempo de execução de cada tarefa.
3. **Cálculo da Recompensa:** Iteramos pelo vetor ordenado mantendo um rastreador do tempo atual percorrido (`sum`). Para cada tarefa processada, incrementamos o tempo atual com a sua duração e adicionamos à resposta final (`total`) a recompensa gerada (`deadline - sum`).

2 The Smallest String Concatenation (Codeforces - 632C)

O problema consiste em receber um conjunto de N strings e determinar qual é a ordem de concatenação de todos os elementos que produz a string resultante lexicograficamente (alfabeticamente) menor possível.

2.1 Solução

A resolução deste problema baseia-se novamente em uma abordagem Gulosa com ordenação customizada, atingindo uma complexidade de $\mathcal{O}(N \log N \cdot L)$, onde L é o tamanho médio das strings. O fluxo de execução obedece aos seguintes passos:

1. **A Falha da Ordenação Tradicional:** Apenas ordenar as strings alfabeticamente não garante a melhor concatenação. Por exemplo, dadas as strings "a" e "ab", a ordenação alfabética comum as manteria como "a", "ab" (resultado: "aab"). Contudo, a concatenação invertida "ab", "a" gera "aba", que é maior. Precisamos de uma regra de transitividade específica.
2. **Função de Comparação Customizada:** Criamos a função `cmp(a, b)` que testa o resultado prático da união de duas strings. Ela verifica diretamente se a concatenação `a + b` é lexicograficamente menor do que `b + a`. Se for, significa que a string `a` deve obrigatoriamente preceder a string `b` na resposta final.
3. **Processamento e Saída:** Fornecemos o vetor de strings e nossa função `cmp` como parâmetros para o `std::sort`. Após o vetor estar perfeitamente arranjado de acordo com a nossa regra, basta iterar imprimindo as strings sequencialmente, sem espaços.